

NR2DT-04: Step 4 — Translate

Series: NR2DT — New Relic to Dynatrace Migration Steps | **Notebook:** 4 of 10 |
Created: April 2026 | **Last Updated:** 08/27/2026

Overview

Goal of this step: convert every NRQL query in the inventory to DQL. Auto-translate HIGH; review MEDIUM; hand-translate LOW. Resolve every gap before any artifact migrates.

Procedural — see **NRLC-02** for the compiler architecture, the 292 patterns, and confidence theory.

Table of Contents

- 1. [What You'll Produce](#)
- 2. [Run the Translator](#)
- 3. [Review MEDIUM Confidence](#)
- 4. [Hand-Translate LOW Confidence](#)
- 5. [Validate Translated DQL](#)
- 6. [Step Exit Criteria](#)

Prerequisites

Requirement	Details
Audience	Migration lead + assigned engineer for this step

Requirement	Details
Completed	NR2DT-03 — Design
Format	Procedural step — use as a runbook; defer to NRLC for depth
NRLC deep dives	NRLC-02 (NRQL → DQL Translation deep dive)

1. What You'll Produce

Artifact	Format	Purpose
<code>translated-queries.csv</code>	CSV	All queries with final DQL + confidence
<code>manual-translations.md</code>	Markdown	LOW-confidence hand-translations with rationale
<code>dql-validation-report.md</code>	Markdown	Tier 1 syntax validation results
<code>translation-coverage.md</code>	Markdown	% of queries translated; remaining gaps

2. Run the Translator

```
# Re-run translation against the latest inventory (Step 1 produced it)
python3 migrate.py compile --file inventory/all-nrql.txt \
  --output translated-queries.csv --report
```

Output columns: `source_nrql`, `translated_dql`, `confidence`, `score`, `notes`, `warnings`, `fixes`.

Sort by confidence ascending — attack LOW first, then MEDIUM, save HIGH for sample-validate.

3. Review MEDIUM Confidence

MEDIUM means the translation is structurally correct but has edge cases worth verifying. Common review checklist:

Pattern	What to verify
<code>COMPARE WITH</code>	Time alignment correct; append produces the expected shape
Subquery via <code>lookup</code>	Lookup target exists as queryable Grail data
<code>SLIDE BY</code>	Sliding window semantics preserved
Nested <code>if()</code> in <code>FACET CASES</code>	Nested DQL emits the right structure
Custom event types	Source mapping (<code>FROM</code> → <code>fetch</code>) correct

Process: mark each MEDIUM as APPROVED or NEEDS-WORK in the CSV. NEEDS-WORK becomes a hand-translation.

4. Hand-Translate LOW Confidence

LOW translations contain TODO markers or are missing entirely. Common causes:

Cause	Action
<code>apdex(field, t:N)</code>	Configure DT Apdex via custom DQL or use Dynatrace Intelligence SLO
<code>funnel(...)</code>	Reformulate as nested <code>append</code> / <code>lookup</code> chain
<code>cohort(...)</code>	No DT equivalent — reformulate the requirement
Custom event type with no source mapping	Add ingest mapping via OpenPipeline first

Document each hand-translation in `manual-translations.md` with the source NRQL, the new DQL, and the rationale.

If a query can't be translated and the requirement can't be reformulated, document it as **deferred** — it joins the gap analysis from Step 1.

5. Validate Translated DQL

Tier 1 syntax validation — confirm every DQL parses against the target tenant:

```
python3 migrate.py compile --validate --file translated-queries.csv
```

The `--validate` flag on `compile` runs parser-level validation; failures attempt auto-fix (`DQLFixer`) and re-validate. Final report shows pass / fail / fixed-then-passed counts.

Tier 2 tenant validation (referenced metrics/entities/buckets exist) runs in Step 8 before each component imports.

6. Step Exit Criteria

G4 — Translation Complete

- [] Every query has a final DQL (auto, reviewed-MEDIUM, or hand-translated)
- [] All translations pass Tier 1 syntax validation
- [] LOW-confidence count is 0 (all triaged)
- [] `manual-translations.md` documents every LOW resolution
- [] Deferred queries documented with deferred-rationale

Next step: NR2DT-05 — Migrate Dashboards & Alerts.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [NewRelic-to-Dynatrace-Migration-Utilities](#), [nrql-engine](#), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).